MUHAMMAD ANAS SHEIKH

AI Software Engineer / Computer Vision Engineer — Real-time Video AI · Model Serving · MLOps
Karachi, Pakistan | +92 335 1253576 | anasshaikh975@gmail.com
linkedin.com/in/anasxshaikhx4 | github.com/Anasshaikh4 | Portfolio

SUMMARY

AI software engineer who builds and runs real-time video AI in production. At Lambda Theta I own the camera-processing service and the GPU model-serving layer of **NEXA**, an AI video-analytics platform watching live CCTV for UK clients, including a Premier League stadium. My work covers platform design, model serving, capacity planning, releases and incident response. Before NEXA, I built model-serving and MLflow infrastructure for **ELM** (Saudi Arabia), including the model registry used by the team behind an Arabic large language model. Google Cloud Certified Associate Cloud Engineer.

EXPERIENCE

LAMBDA THETA — Jul 2024 – Present

Computer Vision Engineer — NEXA *Feb 2026 – Present*

NEXA turns live CCTV into real-time alerts: camera feeds go through AI models on GPU servers, and a rule engine raises alerts with video evidence and a live view. It serves 3 clients, including a 200+ camera Premier League stadium and a central-London retail district, across Google Cloud and on-site UK servers. Team of 10+ engineers.

Platform & architecture

- **Designed and built the camera-processing service at the core of the platform:** one process per camera that reads the stream, picks frames, runs its AI pipelines, tracks people and raises alerts. Its plug-in framework (base class, auto-discovery, authoring guide) now hosts 20+ detection pipelines built by me and 6 other engineers; my safety-gear (PPE) pipeline became the template for every later one.
- **Built the shared GPU model-serving layer:** up to 16 stateless model workers behind one common request/response format. One loaded model serves many cameras, and a pipeline can switch models by changing one config value.
- **Built the alerting path:** one alert per incident instead of one per frame, deduplication per person across pipelines, a video clip from before each incident, and back-pressure so a slow consumer never stalls the AI. Moved alerts from Kafka to Redis Streams with no lost alerts, then retired Kafka from production.
- **Built a model registry on MLflow** with its own logins and roles, staging-to-production promotion with rollback, and an agent on each server that pulls only checksum-verified models. Catalogued all 19 production models and proved backups with a full wipe-and-restore. Now training a junior engineer to take it into production.

Performance & computer vision

- **On a lab server, raised cameras per server 4×, from ~4 to 16** (8 AI pipelines at 3 fps, zero dropped frames) by moving 12 models to NVIDIA Triton + TensorRT (98.5–100% matching detections), decoding video on the GPU and sharing one model call per frame across pipelines. At 6 cameras, GPU load fell from 84% to 30% and latency from 236 to 127 ms. Production rollout planned.
- Found that an unpinned library upgrade had silently changed the pose-estimation model's output in production. Matched the new behaviour in TensorRT and pinned the version, raising agreement from **59.8% to 98.5%**.
- **Set hardware sizing limits by testing each machine until it failed:** 10 cameras per cloud T4 GPU server (limited by CPU) and 8 per NVIDIA Jetson edge device (limited by its hardware video decoder). This replaced guesswork in hardware planning.
- **Made small weapons detectable at CCTV distance** with person-focused tiled detection that zooms in only around people, so empty scenes cost almost nothing. Two more pipelines reuse it.
- **Disproved a suspected "lost detections" defect raised by management.** A repeatable test showed the app and the raw model agree 100% (2,165/2,165). Found and fixed the real cause: a frame-timing bug that delivered up to 22.8% fewer frames than configured.

Reliability, releases & team

- **Found and fixed a 31-hour silent outage:** camera processes were leaking network connections until they hit the operating system's open-file limit, so AI results stopped while every dashboard stayed green. Fixed both leaks and kept production running until the fix shipped.
- Resolved two more production incidents: runaway logging that filled **194 GB** of disk and crashed the database, and a 2-week, 13-camera outage at a remote site that I traced to network routing on the client's side.
- **Shipped release v1.1.0 to all three production servers** as the first production deployments through the team's new GitHub Actions / Cloud Build pipeline. It encrypted stored video evidence, closed the databases to outside access and removed Kafka, with zero data loss.

- **Reviewed pull requests across the engineering team.** Caught 3 production-breaking bugs in a new camera orchestrator (and wrote its 41-check test suite), a login fix that would have allowed unlimited password guesses, and a hard-coded setting that would have slowed every camera to a third of its speed. **Led a security review** that found a high-severity flaw exposing one client's alerts to another.

MLOps Engineer — ELM, Saudi Arabia (offshore team) *Aug 2025 – Dec 2025*

- Deployed and ran the MLflow experiment-tracking server and model registry for ELM's team building an Arabic large language model, which now powers the AI chat assistant in a national Hajj & Umrah app.
- Made model training repeatable and traceable: one config file drives train → evaluate → register, and every model version links back to its data and settings. Demonstrated automated deployment from the registry with GitHub Actions.

Software Engineer — ELM, Saudi Arabia (offshore team) *Oct 2024 – Jul 2025*

- Built the model-serving backend for two vision products: 11 models in 8 containers behind one API gateway. The exam-centre invigilation system (posture, phone use, ID cards) went live in a 10-camera exam centre; people analytics covered faces, gaze and gender. Adding a model became one container plus one config line.
- Trained the classifiers and detectors on lightweight architectures (MobileViT, YOLOv8). Tested accident detection on an NVIDIA Jetson dashcam device and showed it ran ~9× slower than real time, so the model had to change before deployment.

Design Intern *Jul 2024 – Sep 2024*

- Trained and evaluated image-classification models on custom datasets and reported comparative results across architectures.

PROJECTS

- **CCTV threat-detection prototype** (2025): Wrote the architecture spec first, then built a camera service (weapon and pose detection, rule engine, evidence clips, live API) and a React operator dashboard. My lightweight custom detector matched RT-DETR-L on held-out test data (mAP@0.5 **0.936 vs 0.939**) with **8.9× fewer parameters**. I later rebuilt this design at production scale as NEXA's camera service.
- **RAG-as-a-Service** (2026): Multi-user document Q&A platform on FastAPI, PostgreSQL, Milvus, Redis and Celery, with logins and roles, background document processing, and switchable embedding providers (Ollama / OpenAI).
- **Multi-agent SEO analyzer** (2025): 6 AI agents with 11 measurement tools on Google ADK / Gemini, running in parallel.
- **Dental image classifier** (2026): Custom-trained YOLO model behind a Streamlit web app, deployed on Streamlit Cloud.

TECHNICAL SKILLS

Computer vision & ML: PyTorch, YOLO, RT-DETR, RF-DETR, ByteTrack, OpenCV, TensorFlow, scikit-learn
Model serving & MLOps: NVIDIA Triton, TensorRT, ONNX, MLflow, GPU video decoding, gRPC
Backend & data: Python (asyncio), FastAPI, Redis Streams, MongoDB, PostgreSQL, MinIO / S3, Kafka, Milvus, Celery
Video & infrastructure: FFmpeg, RTSP, LiveKit / WebRTC, Google Cloud, GitHub Actions, Docker, systemd, Linux, Nomad, Git
LLM: RAG, OpenAI API, Google ADK / Gemini, Ollama
Hardware (hands-on): Edge: NVIDIA Jetson Thor 128 GB, Jetson Orin Nano 8 GB · Data-centre GPUs: RTX A6000 48 GB, L4 24 GB, Tesla T4 16 GB · Workstation GPUs: RTX 5070 Ti 16 GB, RTX 4070 Ti SUPER 16 GB, RTX 5060 8 GB, RTX 4060 8 GB

CERTIFICATIONS

- **Google Cloud Certified — Associate Cloud Engineer** (Jul 2026 – Jul 2029)
- Build Infrastructure on Google Cloud with Terraform — Google Cloud Skill Badge
- Implement Load Balancing on Compute Engine — Google Cloud Skill Badge
- Supervised Machine Learning: Regression and Classification — Stanford / DeepLearning.AI (Coursera)
- AI Agents Intensive Course — Google (Kaggle)
- Fundamentals of MLOps — KodeKloud
- Computer Architecture — KodeKloud

EDUCATION

DHA Suffa University — BSc Computer Science, GPA 3.29 *Nov 2020 – Jul 2024*